

1. Perforce (Helix Core)

p4d — The Server

p4d	Start the server.
p4d -r <root> -p <port> -d	Server root directory (P4ROOT): holds metadata (db.*) and versioned depot files.
-p <port>	Port to listen on, eg. 1666.
-d	Run as a daemon (background).

```
$ mkdir -p ~/perforce/root
$ p4d -r ~/perforce/root -p 1666 -d
```

Initial Admin Setup

p4 passwd	Set the admin (super user) password. <i>Run this first: with no protections table yet, the connecting user is implicitly super.</i>
p4admin	Admin console — add users, create depots, set protections, etc.

Depots

A **depot** is the top-level container for versioned files on the server — the closest equivalent to a Git “repo”.

local	Default type. Plain versioned files, addressed as //depot/....
stream	Files organized under Streams (managed branching / merging).

2. p4 — The Client

Configuration: Env Vars

P4PORT	host:port of the p4d server. Eg: localhost:1666.
P4USER	Perforce username.
P4CLIENT	Name of the workspace (client) to use.
P4EDITOR	Editor used for forms (submit, change, client, ...). Eg: vim.
P4DIFF	External diff program used by p4 diff. Eg: vimdiff.

.p4config files

P4CONFIG names a config file (eg. .p4config). p4 searches **upward** from the current directory for a file with that name and loads KEY=VALUE settings from the first one found. Lets each directory tree pin its own port/user/client.

```
# .p4config
P4PORT=localhost:1666
P4USER=bob
P4CLIENT=bob-ws
P4EDITOR=vim
P4DIFF=vimdiff
```

Loading P4CONFIG

bash	export P4CONFIG=.p4config Add to ~/.bashrc to persist.
PowerShell	\$env:P4CONFIG = ".p4config" Persist: notepad \$PROFILE. [Environment]::SetEnvironmentVariable("P4CONFIG", ".p4config", "User")

p4 set

p4 set	List all settings and where each one comes from.
p4 set <VAR>=<val>	Persist a setting in the P4ENVIRO file (~/.p4enviro; registry on Windows).
p4 set <VAR>=	Unset a setting.

Precedence (highest first): command-line flags > P4CONFIG file > env vars > p4 set (P4ENVIRO) > defaults.

p4 info

Shows the current connection / workspace state. Key fields to check:

Client name	Workspace in use (P4CLIENT).
Client root	Local directory the client maps to.
Current directory	cwd that p4 sees.

Scope of Commands

- Commands like p4 status (no path args) only act on the **subtree rooted at the current directory**.
- The current directory must be **inside the client root**. If cwd is outside the client root, these commands find nothing / fail.

3. p4 — Working with Files

Note: ... is a recursive wildcard matching all files under a path (eg. p4 revert ... = everything under cwd).

Sync

p4 sync [path...]	Updates the workspace to match the depot, per the client view (like git pull for files). With no path, acts on the subtree under cwd.
path#<rev>	Sync to a specific revision.
path@<cl>/label>	Sync to the state as of a changelist, label, or date.
-f	Force: re-transfer files even if already at that revision (eg. to repair local edits made outside p4).
-n	Preview only — no files transferred.

Open for Change

p4 edit <file>	Open an existing depot file for edit (checkout).
p4 add <file>	Mark a new (untracked) file for add.
-c <cl>	Open in pending changelist <cl> instead of default. Applies to edit, add, revert, reopen, shelve, unshelve, submit.

Reconcile

p4 reconcile [-a] [-e] [-d] [-c <cl>] [path...]	Detects changes made outside p4 and opens files accordingly. No flags = all of -a -e -d.
-a	Open files added on disk for add.
-e	Open modified files for edit.
-d	Open missing files for delete.

Revert / Reopen

p4 revert [-a] [-c <cl>] <file>	Discard pending changes, restore depot version.
-a	Only revert files <i>unchanged</i> from depot (safe revert).
p4 reopen -c <cl> <file>	Move opened file(s) to a different pending changelist.

Shelve / Unshelve

p4 shelve -c <cl> [file]	Save files opened in <cl> to the server without submitting.
-d	With shelve: discard the shelved file(s) from <cl>.
p4 unshelve [file]	Restore shelved files into a pending changelist.
-s <cl>	Source: changelist to unshelve from.
-c <cl2>	Target: changelist to receive the files (default default).

Submit

p4 submit -c <cl>	Submit pending changelist <cl> to the depot.
-------------------	--

Diff

p4 diff [**file...**]

Compare the file **on disk** against a depot revision. Runs **P4DIFF** (eg. **vimdiff**) if set. Default: opened files vs the synced (**have**) revision.

p4 diff file	Disk vs have revision.
p4 diff file#2	Disk vs revision #2.
p4 diff file@<c1>	Disk vs the file as of changelist <c1>.
-f	Diff files not opened for edit.

p4 diff2 <f1> <f2>

Diff two **depot** revisions. Runs **on the server**, so **P4DIFF** is ignored — output is a plain text diff.

p4 diff2 f#1 f#4	Between two revisions of a file.
p4 diff2 f@<c1> f@<c2>	Same file across two changelists.
p4 diff2 f1 f2	Two different depot files.
-u	Unified diff format.
-q	Quiet: omit identical-file messages.

To use **vimdiff** on two depot revisions, print them to temp files:

```
$ p4 print -q file#1 > /tmp/a
$ p4 print -q file#4 > /tmp/b
$ vimdiff /tmp/a /tmp/b
```

Revision Specifiers

file#<n>	Revision n.
file#head	Latest depot revision.
file#have	Revision last synced into the workspace.
file@<c1>	Revision as of changelist <c1> (also takes a label or date).
file@=<c1>	Revision shelved or submitted in exactly <c1>.

Inspecting

p4 opened	List opened (checked-out) files.
-c <c1>	Filter by changelist.
-u <user>	Filter by user.

p4 filelog <file>	Show a file's revision history.
--------------------------------	---------------------------------

p4 changes	List changelists.
-u <user>	Filter by user.
-m <n>	Limit output to n changelists.
-s <status>	Filter by status: pending / submitted / shelved .